

Tarea 1 Cadenas de Markov

Aplicación del muestreo de Gibbs a los modelos Hard-Core y q-coloraciones

Curso: Cadenas de Markov y Aplicaciones (2025-II)

Profesor: Freddy Hernández-Romero

Integrantes del grupo:

- Juan Bernardo Soto Pescador
- Francisco José Gutiérrez Peña
- Tuli Peña Melo

Punto 1: Muestreo de Gibbs para el Modelo Hard-Core

El objetivo es muestrear configuraciones de partículas en una rejilla $K \times K$ bajo la restricción **hard-core**: dos partículas no pueden ocupar celdas adyacentes (vecinos ortogonales). Se implementa un Gibbs sampler que actualiza una coordenada (celda) por paso, ya sea seleccionada aleatoriamente o siguiendo un orden, y decide colocar/retirar una partícula según la factibilidad (vecinos vacíos) y una probabilidad que garantiza la distribución estacionaria.

La hipótesis explorada en el notebook es que el número típico de partículas en la distribución estacionaria es aproximadamente $K^2/4$ para rejillas grandes.

Código implementado

Solo mostramos los últimos bloques de código. El código completo puede verse en el Notebook.

```
final_time = 100001
chain_times = [500, 5000, 10000, 10001, 10002, 100000]

# Run Gibbs random sampler
def get_gibbs_samples_random(chain_times):
    final_grids = []
    for k in K_list:
        print(f"For K={k} we have:")
        updated_grid, updated_grid_2 = create_k_matrix(k)
        for t in range(final_time):
```

```

        updated_grid = run_gibbs_random(updated_grid, k)
        if t in chain_times:
            final_grids.append(updated_grid)
            print(f"X_{t} grid:\n{updated_grid}")

    return final_grids

x = get_gibbs_samples_random(chain_times)

```

```

# Run gibbs w order sampler
chain_times = [500, 5000, 10000, 10001, 10002, 100000]
i = 0
j = 0
def get_gibbs_samples_order(chain_times, i, j):
    final_grids = []
    for k in K_list:
        print(f"For K={k} we have:")
        updated_grid, updated_grid_2 = create_k_matrix(k)
        for t in range(final_time):
            updated_grid, i, j = run_gibbs_order(updated_grid, k, i, j)
            if t in chain_times:
                final_grids.append(updated_grid)
                print(f"X_{t} grid:\n{updated_grid}")

    return final_grids

x = get_gibbs_samples_order(chain_times, i, j)

```

```

def count_particles(grid) -> int:
    """Return the number of particles (cells equal to 1) in the
    grid."""
    arr = np.asarray(grid)
    return int(np.count_nonzero(arr == 1))

def compute_mean_particles_ordered(runs: int, K_list, chain_times,
final_time: int):
    """Return {K: {t: mean_num_particles}} across `runs` for ordered
    Gibbs."""
    means_by_k = {}
    for K in K_list:
        # accumulator: t -> list of counts
        counts_per_t = {t: [] for t in chain_times}

```

```

        for _ in range(runs):
            grid, _grid_ones = create_k_matrix(K)
            i, j = 0, 0
            for step in range(final_time):
                grid, i, j = run_gibbs_order(grid, K, i, j)
                if step in counts_per_t:
                    counts_per_t[step].append(count_particles(grid))
            # convert to means
            means_by_k[K] = {t: float(np.mean(counts)) if counts else 0.0
                             for t, counts in counts_per_t.items()}
        return means_by_k

def plot_mean_particles_hist(means_by_k):
    """Plot bar charts of mean particles at each chain time for every
    K."""

    # Determine chain_times order from the first K entry
    sample_K = next(iter(means_by_k))
    chain_times_sorted = sorted(means_by_k[sample_K].keys())

    num_K = len(means_by_k)
    cols = 2
    rows = (num_K + cols - 1) // cols
    plt.figure(figsize=(6 * cols, 3.5 * rows))

    for idx, (K, means_dict) in enumerate(sorted(means_by_k.items())):
        plt.subplot(rows, cols, idx + 1)
        values = [means_dict[t] for t in chain_times_sorted]
        plt.bar([str(t) for t in chain_times_sorted], values,
color="#4C78A8")
        plt.title(f"K={K} mean particles by chain time")
        plt.xlabel("chain time")
        plt.ylabel("mean # particles")
        plt.tight_layout()

    plt.show()

```

```

# Configure parameters and run aggregation
runs = 10 # number of independent runs
chain_times = [10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]
final_time = 100001

```

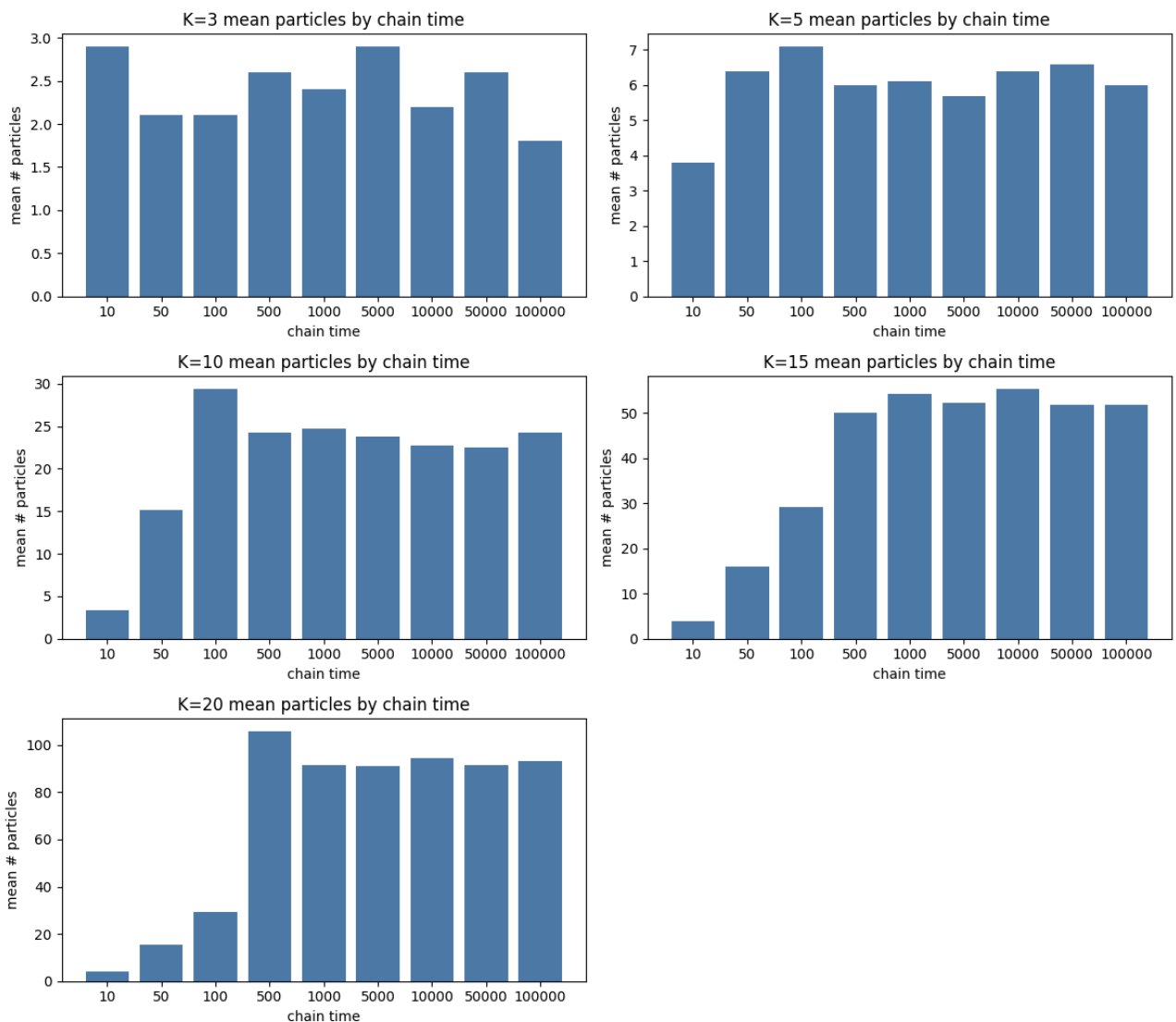
```

means_by_k = compute_mean_particles_ordered(runs, K_list, chain_times,
final_time)
print("Mean #particles by K and chain time:")
for K in sorted(means_by_k):
    print(f"K={K}:", {t: round(means_by_k[K][t], 2) for t in
sorted(means_by_k[K])})

plot_mean_particles_hist(means_by_k)

```

Resultados



Medias del número de partículas por K y tiempo de cadena (salida del notebook):

K=3: {10: 2.9, 50: 2.1, 100: 2.1, 500: 2.6, 1000: 2.4, 5000: 2.9, 10000: 2.2, 50000: 2.6, 100000: 1.8}

$K=5$: {10: 3.8, 50: 6.4, 100: 7.1, 500: 6.0, 1000: 6.1, 5000: 5.7, 10000: 6.4, 50000: 6.6, 100000: 6.0}
 $K=10$: {10: 3.4, 50: 15.1, 100: 29.4, 500: 24.3, 1000: 24.7, 5000: 23.8, 10000: 22.7, 50000: 22.5, 100000: 24.3}
 $K=15$: {10: 4.0, 50: 15.9, 100: 29.3, 500: 50.0, 1000: 54.3, 5000: 52.3, 10000: 55.4, 50000: 51.9, 100000: 51.9}
 $K=20$: {10: 3.9, 50: 15.5, 100: 29.2, 500: 105.8, 1000: 91.5, 5000: 91.1, 10000: 94.5, 50000: 91.5, 100000: 93.0}

La figura muestra, para cada valor de K (3,5,10,15,20), una barra por cada `chain_time` con la media del número de partículas estimada a partir de varias corridas independientes. Se observa que para tiempos grandes la media se estabiliza aproximadamente alrededor de un valor próximo a $K^2/4$ (ver tabla arriba). Para rejillas pequeñas ($K=3$) hay más variabilidad por el efecto de borde.

Conclusiones

Observando las medias por `chain_time` (10,50,...,100.000) se aprecia que para $K=10,15,20$ las medias en los tiempos mayores (5,000–100,000) se estabilizan cerca de 22–25 ($K=10$), 50–55 ($K=15$) y 90–94 ($K=20$). Estos valores están cerca de $K^2/4$: para $K=10$, $100/4=25$; $K=15$, $225/4$ (approx 56.25); $K=20$, $400/4=100$. Por tanto la conjetura es que la cadena converge hacia una distribución cuyos recuentos típicos de partículas están alrededor de $K^2/4$.

A juzgar por las medias en `chain_time` crecientes y la estabilización observada, hay indicios de convergencia para las rejillas y parámetros probados, aunque la velocidad de convergencia depende de K . Para K grandes se requieren más pasos (el notebook sugiere usar 50,000 o más para rejillas grandes).

Punto 2: Muestreo de Gibbs para q -coloraciones

Se implementa un Gibbs sampler para generar coloraciones propias de una rejilla $K \times K$ con q colores (cada celda toma un color en $\{1..q\}$ tal que celdas vecinas no compartan color). El muestreador actualiza una celda por paso condicionando en los colores de las demás; dado que todas las coloraciones propias son equiprobables en el espacio objetivo, el Gibbs sampler produce una distribución uniforme sobre coloraciones propias.

Código implementado

Solo mostramos una parte del código. El código completo puede verse en el Notebook.

```
def create_grid(K, q):
    """
```

Crea una rejilla KxK con una coloración inicial válida.
Usa un patrón de tablero de ajedrez con dos colores para garantizar una coloración propia inicial.

```
"""  
grid = np.zeros((K, K), dtype=int)  
for i in range(K):  
    for j in range(K):  
        # Patrón de tablero de ajedrez usando los primeros dos  
colores  
        grid[i, j] = (i + j) % min(2, q)  
return grid  
  
def get_neighbors(i, j, K):  
    """  
    Obtiene los vecinos válidos de una posición (i, j) en la rejilla.  
    """  
    neighbors = []  
    # Arriba  
    if i > 0:  
        neighbors.append((i-1, j))  
    # Abajo  
    if i < K-1:  
        neighbors.append((i+1, j))  
    # Izquierda  
    if j > 0:  
        neighbors.append((i, j-1))  
    # Derecha  
    if j < K-1:  
        neighbors.append((i, j+1))  
    return neighbors  
  
def get_valid_colors(grid, i, j, q):  
    """  
    Obtiene los colores válidos para una posición (i, j) dada la  
configuración actual.  
    Un color es válido si no está presente en ningún vecino.  
    """  
    K = len(grid)  
    neighbors = get_neighbors(i, j, K)  
  
    # Colores usados por los vecinos  
    used_colors = set()  
    for ni, nj in neighbors:
```

```

        used_colors.add(grid[ni, nj])

    # Colores disponibles
    valid_colors = [c for c in range(q) if c not in used_colors]
    return valid_colors

def gibbs_step_q_coloration(grid, q):
    """
    Realiza un paso del Gibbs sampler para q-coloraciones.
    """
    K = len(grid)
    # Seleccionar una posición aleatoria
    i = random.randint(0, K-1)
    j = random.randint(0, K-1)

    # Obtener colores válidos para esa posición
    valid_colors = get_valid_colors(grid, i, j, q)

    if valid_colors:
        # Elegir un color válido uniformemente al azar
        new_color = random.choice(valid_colors)
        grid[i, j] = new_color
        # Si no hay colores válidos (no debería pasar con q >= 3), mantener
        # el color actual

    return grid

def gibbs_sampler_q_coloration(K, q, num_steps, save_steps=None):
    """
    Ejecuta el Gibbs sampler para q-coloraciones.

    Args:
        K: Tamaño de la rejilla (K x K)
        q: Número de colores
        num_steps: Número de pasos de la cadena
        save_steps: Lista de pasos en los que guardar el estado

    Returns:
        grid: Estado final de la rejilla
        saved_states: Estados guardados en los pasos especificados
    """
    grid = create_grid(K, q)
    saved_states = {}

```

```

if save_steps is None:
    save_steps = []

for step in range(num_steps):
    grid = gibbs_step_q_coloration(grid.copy(), q)

    if step in save_steps:
        saved_states[step] = grid.copy()

return grid, saved_states

```

```

# Test básico
K = 5
q = 3
grid = create_grid(K, q)
print(f"Rejilla inicial {K}x{K} con {q} colores:")
print(grid)

```

```

def plot_q_coloration(grid, q, title="q-coloración"):
    """
    Visualiza una q-coloración de la rejilla.
    """
    fig, ax = plt.subplots(figsize=(6, 6))

    # Crear un mapa de colores personalizado
    if q <= 10:
        colors = ['white', 'red', 'blue', 'green', 'yellow', 'purple',
                  'orange', 'pink', 'brown', 'gray']
        cmap = mcolors.ListedColormap(colors[:q])
    else:
        cmap = plt.cm.get_cmap('tab20', q)

    im = ax.imshow(grid, cmap=cmap, vmin=0, vmax=q-1)

    # Agregar líneas de rejilla
    for i in range(len(grid) + 1):
        ax.axhline(i - 0.5, color='black', linewidth=1)
        ax.axvline(i - 0.5, color='black', linewidth=1)

    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_title(title)

```



```

# Agregar barra de colores
cbar = plt.colorbar(im, ax=ax, ticks=range(q))
cbar.set_label('Color', rotation=270, labelpad=15)

plt.tight_layout()
plt.show()

def plot_trajectory_q_coloration(K, q, num_steps, steps_to_show):
    """
    Visualiza la trayectoria del Gibbs sampler mostrando estados en
    diferentes pasos.
    """
    grid, saved_states = gibbs_sampler_q_coloration(K, q, num_steps,
steps_to_show)
    saved_states[num_steps] = grid # Agregar el estado final

    n_plots = len(steps_to_show) + 1
    cols = min(3, n_plots)
    rows = (n_plots + cols - 1) // cols

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 5*rows))
    if n_plots == 1:
        axes = [axes]
    else:
        axes = axes.flatten()

    # Crear mapa de colores
    if q <= 10:
        colors = ['white', 'red', 'blue', 'green', 'yellow', 'purple',
                  'orange', 'pink', 'brown', 'gray']
        cmap = mcolors.ListedColormap(colors[:q])
    else:
        cmap = plt.cm.get_cmap('tab20', q)

    for idx, (step, state) in enumerate(sorted(saved_states.items())):
        if idx < len(axes):
            ax = axes[idx]
            im = ax.imshow(state, cmap=cmap, vmin=0, vmax=q-1)

            # Agregar líneas de rejilla
            for i in range(K + 1):
                ax.axhline(i - 0.5, color='black', linewidth=0.5)

```

```

ax.axvline(i - 0.5, color='black', linewidth=0.5)

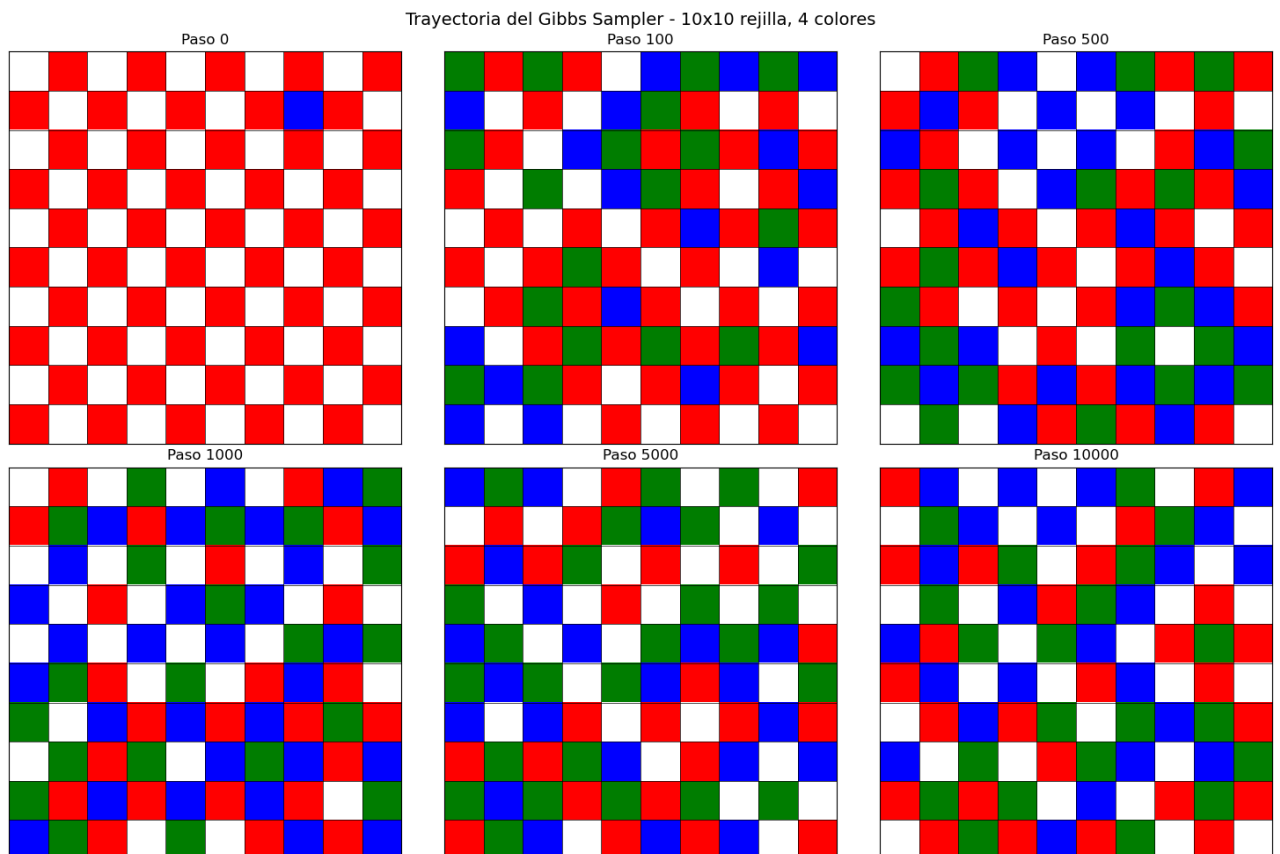
ax.set_xticks([])
ax.set_yticks([])
ax.set_title(f"Paso {step}")

# Ocultar ejes no utilizados
for idx in range(len(saved_states), len(axes)):
    axes[idx].axis('off')

fig.suptitle(f"Trayectoria del Gibbs Sampler - {K}x{K} rejilla, {q} colores", fontsize=14)
plt.tight_layout()
plt.show()

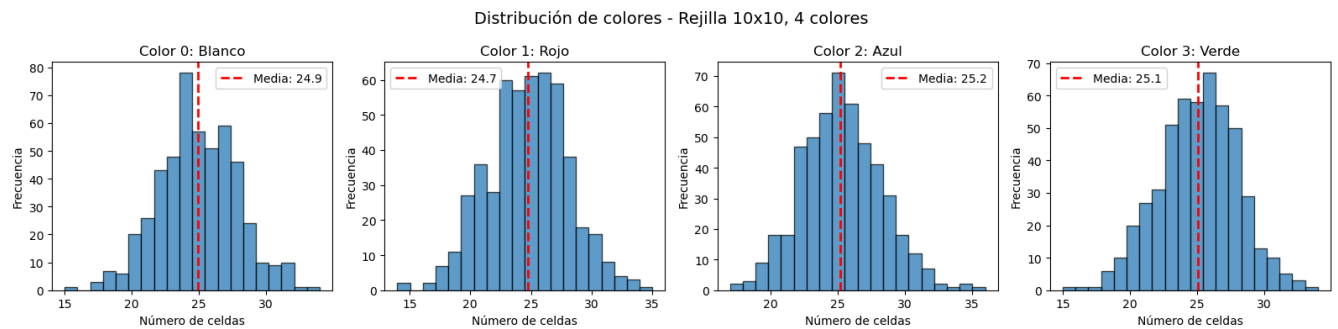
```

Resultados



Trayectoria de la coloración de 10x10 con 4 colores

En los pasos 0, 100, 500, 1000 y 5000. Muestra cómo la configuración se va mezclando y pierde dependencia con la condición inicial.



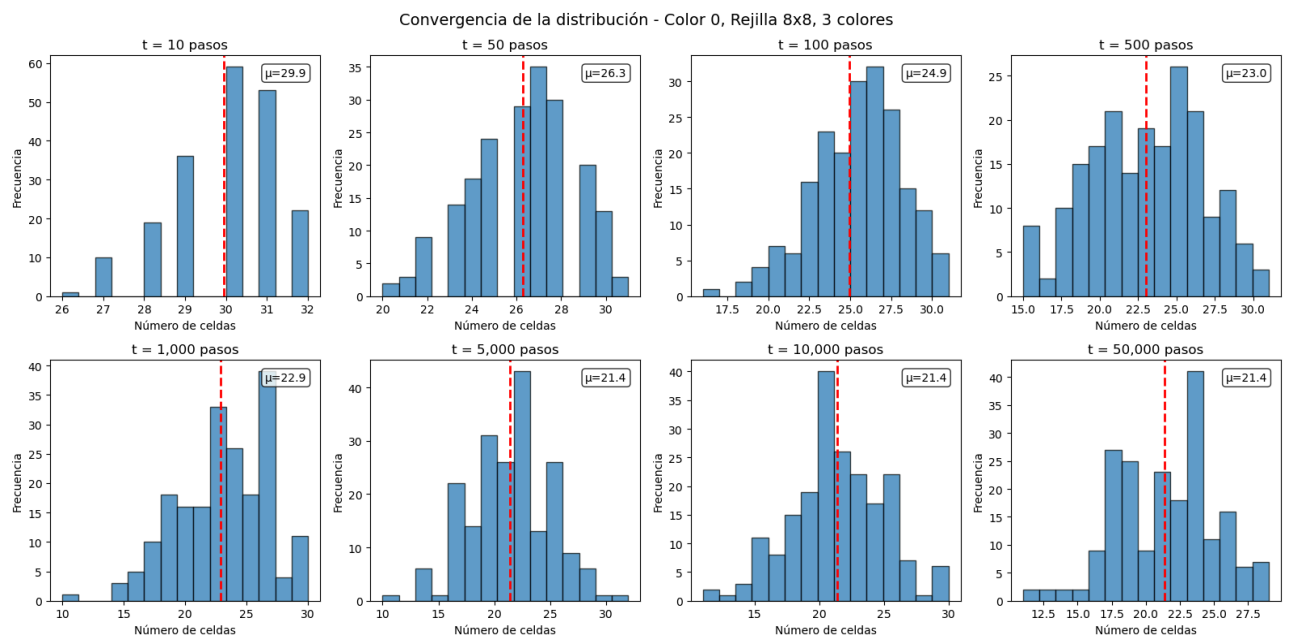
Distribución por colores (de la coloración de 10x10 con 4 colores)

Estadísticas para rejilla 10x10 con 4 colores:

Blanco - Media: 24.94, Std: 3.07, Min: 15, Max: 34
 Rojo - Media: 24.73, Std: 3.15, Min: 14, Max: 35
 Azul - Media: 25.24, Std: 3.12, Min: 17, Max: 36
 Verde - Media: 25.10, Std: 3.09, Min: 15, Max: 34

Número total de celdas: 100

Número esperado por color (uniforme): 25.00



Distribución de frecuencia del color 0 (de la coloración de 8x8 con 3 colores)

La figura muestra la evolución del número de celdas con el color 0 en una rejilla 8x8 con 3 colores a lo largo del muestreo de Gibbs. Cada histograma corresponde a distintos tiempos ($t = 10$ a 50 000 pasos) y evidencia cómo la distribución, inicialmente centrada alrededor de 30 celdas, se desplaza y estabiliza cerca de 21 celdas, el valor esperado teórico ($64/3$). A partir de unos pocos miles de pasos, la media deja de cambiar significativamente y parece estabilizarse en 21.4, indicando que la cadena ha alcanzado la convergencia hacia la distribución estacionaria.

Conclusiones

Los resultados muestran que el Gibbs Sampler para q-coloraciones converge hacia una distribución uniforme sobre coloraciones propias, donde cada color ocupa aproximadamente la misma proporción del total de celdas. En el caso mostrado (rejilla 8×8 , 3 colores), la media del número de celdas por color se estabiliza en torno a 21, cercano al valor teórico $64/3$. Los histogramas y trayectorias indican que la cadena pierde rápidamente la dependencia de la condición inicial y alcanza el equilibrio después de unos pocos miles de pasos ($\approx 5,000$). A partir de ese punto, las distribuciones se mantienen estables y simétricas, lo que sugiere una buena mezcla del muestreador. En general, el comportamiento observado respalda la hipótesis de que el Gibbs sampler converge hacia la distribución estacionaria esperada, con conteos medios de celdas cercanos a K^2/q y una varianza pequeña que disminuye con el tamaño de la rejilla.